

## **Broken Object Level Authorization**

# Introduction

## What is Authorization ?

Authorization is the process of giving someone the ability to access a resource.

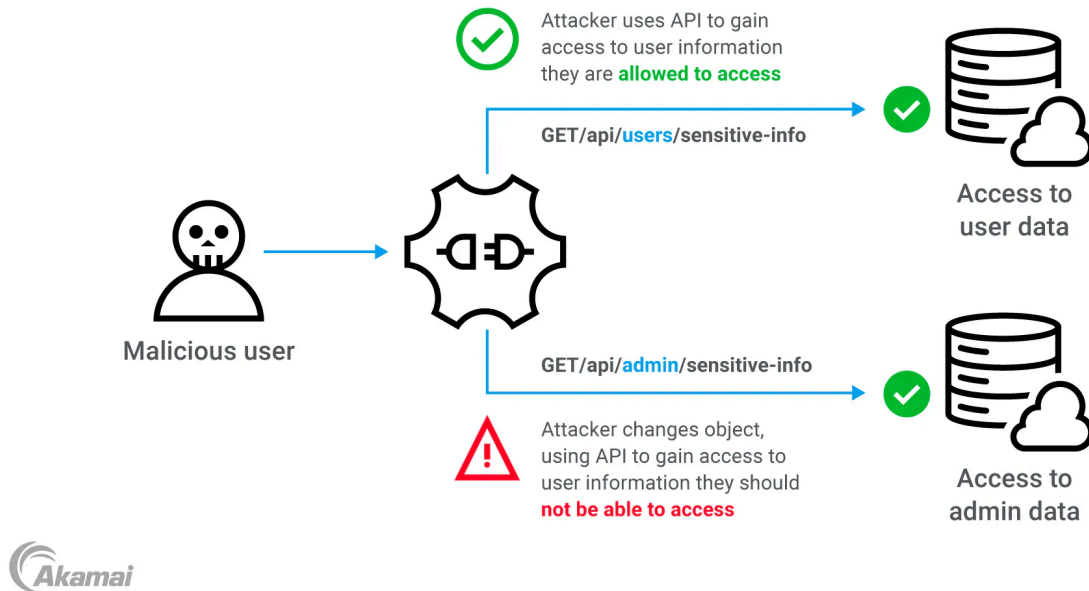
Authorization is deciding who can or cannot access certain things like files, information or features based on rules or permission that have been set beforehand.

## What is BOLA in API Testing ?

When an API provides access to data objects based on the user's role but fails to verify if the user is authorized to access those specific data objects.

The BOLA vulnerability is a flaw in the access control mechanism of a web application, which allows users to access or manipulate "objects" such as database records, files, or other resources within an IT environment. When the application's mechanism doesn't properly enforce access controls, unauthorized users are allowed to perform actions on objects that they should not have permission to access. - According to Akamai

## What is Broken Object Level Authorization (BOLA)?



## What is the impact of this issue ?

BOLA Vulnerability can have several serious impacts such as

- Unauthorized access that can result in data disclosure to 3rd parties.
- Data Loss and Data Manipulation
- Unauthorized access to objects can also lead to full account takeover.

## Example

When a user logs into their account and sends a get request to the site, for example: <https://example.com/api/v3/users?id=1342> and receives a response which contains their own properties (user object properties), then that is a normal scenario.

```
{  
  "id": 1342,  
  "Name": "Bob",  
  "DOB": "6-7-2001",  
  ...  
}
```

However, if the user is able to change the user id to something else such as 2245, i.e <https://example.com/api/v3/users?id=2245> and is able to access some other user's object properties, then the application is known to be vulnerable to BOLA.

# Testing Methodology

One can test for BOLA vulnerability by properly understanding the url and then changing the user object identifier (key) which are usually **IDs**.

For example:

GET /api/user/**1** <- 1 is the user id.

GET /admin/settings/account/**username1** <- username1 is the user id.

These IDs can be changed to something else to check for BOLA.

## Types of BOLA

- **Predictable ID**
  - Valid request: /api/account/1001
  - BOLA request: /api/account/1002
- **ID Combo**
  - Valid request: /api/v1/user1/data/1001
  - BOLA request: /api/v1/user2/data/1002
- **Email as user ID**
  - Valid request: GET /api/account {"email": "[userA@gmail.com](mailto:userA@gmail.com)"}
  - BOLA request: GET /api/account {"email": "[userB@gmail.com](mailto:userB@gmail.com)"}
- **Multiple objects**
  - POST /api/v1/user/checking {"Account": 1222}
  - POST /api/v1/user/checking {"Account": 1222, "Account": 3333}

## Conclusion

Broken Object Level Authorization (BOLA) is a critical vulnerability arising from lack of authorization checks on data (usually user) objects accessed via APIs. The issue exists in application's failure to verify if a user is truly authorized to interact with a specific object, even if they have a valid role, allowing attackers to access or modify resources belonging to other users.

The potential impact of a BOLA vulnerability is severe, ranging from unauthorized data disclosure and manipulation to full account takeover.

Preventing BOLA requires a deep understanding of how object identifiers are used in API requests, whether in the URL path, query parameters, or request body, and implementing strict, per-user access control checks on every function that accesses a data object. Rigorous testing for BOLA, often by systematically altering ID parameters, is an essential part of securing any modern API architecture.

## References

<https://www.akamai.com/glossary/what-is-bola>

<https://developers.cloudflare.com/api-shield/security/bola-vulnerability-detection/>

